Kinesis Design Document

William N. Johnson

Version 0.5.4 (in development), October 2026

Abstract

Kinesis is a desktop application for keeping and studying the videos you learn from. It saves YouTube videos with their transcripts, lets you summarize, tag, file and link them, and lets you plan them into routines such as a fitness program or a school course. This document describes how the application is put together: its architecture, its data model, the main features and the reasons behind the less obvious decisions. It is written for people who will work on the code.

Contents

1 What Kinesis is for **4**

1.1 Goals 4

1.2 Non-goals 4

2 Architecture **4**

2.1 The window 6

3 Workspaces and storage **7**

3.1 Workspaces 7

3.2 The database 8

3.3 Main tables 9

3.4 Attachments 9

4 The Drive system **10**

4.1 Two spellings 10

4.2 Homes and links 10

4.3 Changing the tree 10

4.4 Sequences 10

5 Getting videos in **11**

6 Search and filtering **11**

7 The sidebar **12**

8 Keeping up with YouTube **12**
8.1 Renamed channels . . . 13
8.2 Renamed and removed videos . . . 13
8.3 Orphaned videos . . . 14

9 Links between things **15**

10 Glossary and Biography **15**
10.1 Biographies . . . 15

11 AI summaries and images **16**

12 The Calendar **17**
12.1 Tracks . . . 17
12.2 Patterns . . . 18
12.3 The week . . . 19
12.4 Time windows . . . 19
12.5 Starting over . . . 20
12.6 Deleted videos . . . 20
12.7 Export . . . 21

13 Pins **21**

14 Notes **21**
14.1 The page . . . 22
14.2 Citing . . . 24
14.3 On the Calendar . . . 25
14.4 Workbooks . . . 26
14.5 Personal and shared . . . 27
14.6 Data . . . 27

15 Logs **27**

16 Graph view **29**

17 Deleting and the Trash **29**

18 Permissions **30**

19 Sharing and export **31**
19.1 Kinpak . . . 31

19.2 Obsidian 31

20 Sync **31**

21 Interface conventions **32**

22 Testing **32**

23 Open work **33**

1 What Kinesis is for

Most people who learn from video keep track of it badly. A useful lecture ends up in a browser bookmark, the point they wanted to remember is somewhere in minute 40, and the connection between that lecture and an article they read last month exists only in their head. Kinesis treats a video as a document. Once saved, it has a transcript you can search, a summary you can edit, tags and glossary terms, notes, attached files, and links to and from other videos, terms and people.

The second half of the application is about doing something with that library over time. Tracks on the Calendar turn a list of videos and tasks into a plan: one item after another, a weekly schedule like a class, or a repeating rotation like a workout split.

1.1 Goals

- Everything lives on the user's machine, in a SQLite database they can copy, back up or move to an external drive. No account is needed to use the app.
- A workspace can be handed to someone else as a single file, without leaking API keys or local paths.
- The person who sets up a workspace (a teacher, a research lead) can decide what its readers see and can change.
- Large libraries stay usable. The maintainer's own library has more than eight thousand items, and features are tested against that size.

1.2 Non-goals

- Kinesis does not host or download video. Playback is YouTube's embedded player.
- Sync clients never upload. A sync server publishes curated content; it does not collect anything back.
- The project is in beta. The sync protocol and the Kinpak format may change between versions, and there are deliberately no compatibility shims for older formats.

2 Architecture

Kinesis is a Tauri 2 application. The backend is Rust (edition 2021, minimum Rust 1.88) and the user interface is React 19, built with Vite 7 and styled with Tailwind CSS 4. The window has no native title bar; the app draws its own. It can be made no smaller than 800 by 600, the smallest of the sizes Settings offers under Display, so it cannot be dragged below what it can be set to; rows fold as it narrows rather than squeeze. The horizontal layout's sections (Search, Library, Glossary and the rest) are a row of names, the open one white with a line in the accent color under it, so they read as one group and not as separate buttons; the ones that no longer fit move, from the end, into *More*, as the sidebar's tabs do.

Layer	Responsibility
`src/` (TypeScript)	Every screen, plus `api.ts`, the one place that calls backend commands. A large feature keeps its parts in a folder of its own (`src/components/calendar`), common controls live in `src/components/ui`, and self-contained feature logic (pins, bulk assign, returning to the graph) lives in hooks under `src/hooks`.
`src-tauri/src/commands`	Tauri commands. Each runs its database work on a blocking thread so a slow query never freezes the window.
`src-tauri/src/db`	All SQL, one module per area (videos, glossary, calendar, pins and so on).
`src-tauri/src/youtube`, `venice.rs`	Talking to YouTube and the AI provider.
`src-tauri/src/sync`	The sync client engine.
`crates/kinesis-sync-proto`	Wire types, the settings allowlist, hashing and the pack codec, shared by the client, the reference server and the Kinpak format.
`crates/kinesis-sync-server`	The reference sync server.

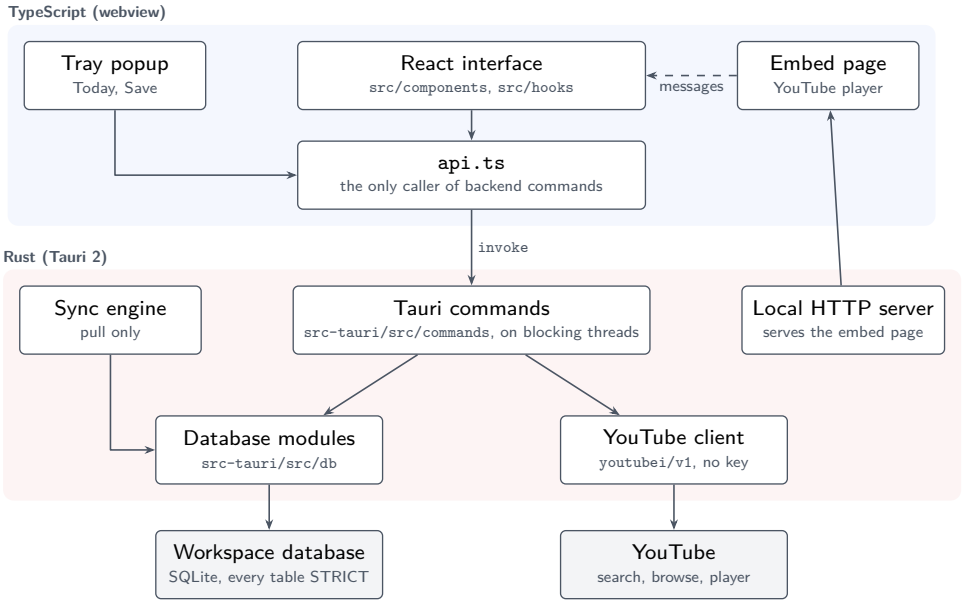


Figure 1: How the parts talk to each other. Everything the interface asks of the backend goes through `api.ts`; the embed page answers the sidebar with window messages (a seek, the current time, a video that played to its end).

A few smaller pieces run beside the main window:

- A small local HTTP server serves the YouTube embed page on `localhost`. The video ID it is given is checked against YouTube's 11-character alphabet before it is placed in the page, which is what keeps it from being an injection point.
- On Windows, a tray icon opens a small frameless popup with two tabs. *Today* lists the Calendar's items for today, and anything overdue, with checkboxes. *Save* takes a YouTube

link, filled in from the clipboard, and saves it to the Library. The popup opens on Today unless a new link is on the clipboard. With *Keep running in the tray* on, closing the main window hides it instead of quitting.

- Every way of leaving the app (the close button, Alt+F4, the tray's Quit) first asks the page whether something is unsaved. A page that does not answer within a short timeout does not keep the app open.
- Window size, position, layout and navigation style are per machine, stored in a JSON file in the config directory. Everything else is per workspace, in that workspace's database.

2.1 The window

Figure 2 shows the main window on the Library. The title bar 1 holds the section tabs (a workspace can rename them) and, on the right, the pin menu 2, whose number counts the Calendar items due today. The search bar 3 turns typed prefixes into facet chips (Section 6). The Drive panel 4 filters the grid to one branch of the tree. A card whose thumbnail comes back as YouTube's gray placeholder 5 has most likely been removed from YouTube; Section 8 explains what the app does about it. Every section ends in a status bar 6; the Library's shows the result count, the Orphaned and Trash chips, and the sort controls.

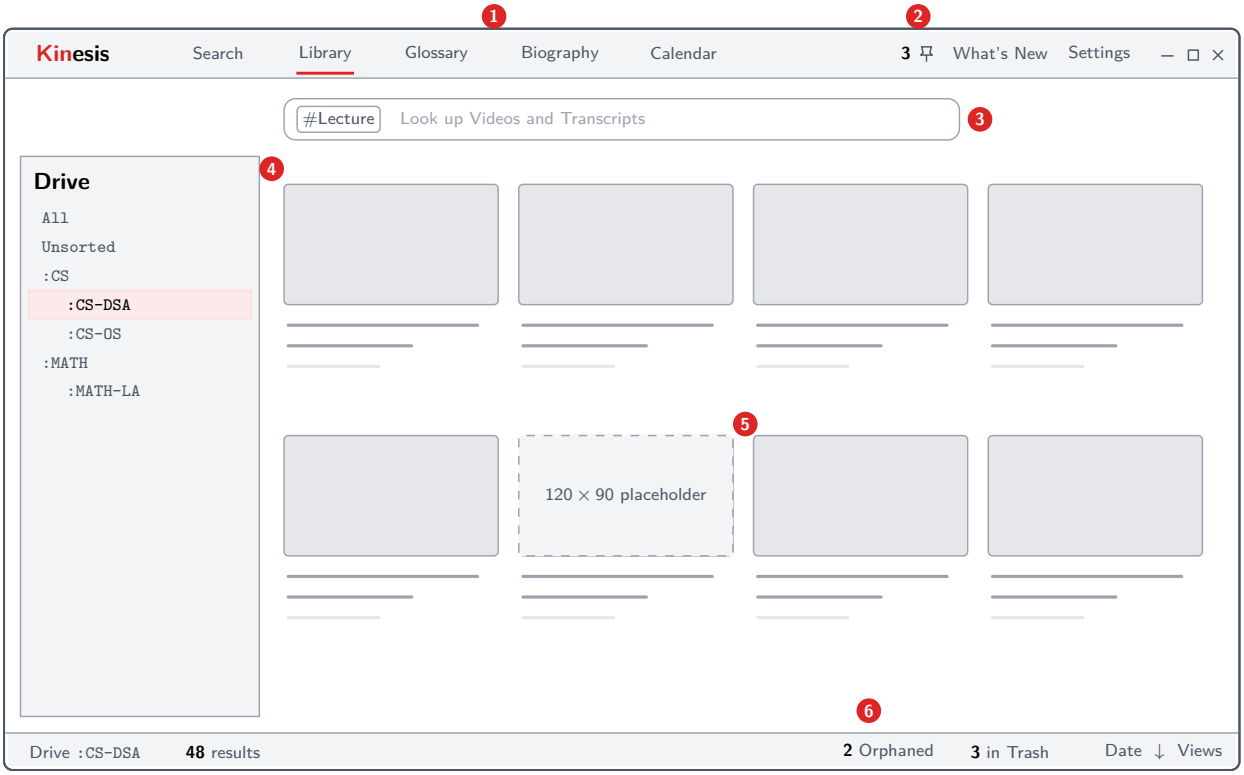


Figure 2: The main window on the Library, with a Drive selected and a facet chip in the search bar.

3 Workspaces and storage

3.1 Workspaces

A workspace is a folder under the application's data directory. It holds an `init.conf` and usually the database, `kinesis_data.db`. If the database lives somewhere else, for example on an external drive, the folder holds only the `init.conf`, which points at it:

```
kinesisapp/
  Computer Science Workspace/
    kinesis_data.db
    init.conf db_path: E:\Drives\CS (only when the database lives elsewhere)
                  last_opened: 1789000000000
```

There is no registry of workspaces. The recent list is a scan of these folders, and the most recent workspace is the one with the newest `last_opened`. A workspace whose drive is unplugged stays in the list, dimmed, with a *Locate* action. An exclusive lock file beside the database stops two Kinesis windows from opening the same workspace.

The Workspaces screen (Figure 3) is that scan made visible. The open workspace is marked with the accent bar 1; one on an unplugged drive shows *Not found* and *Locate* 2. The main area offers the three ways to add a workspace 3: start an empty one, import a Kinpak, or point at a folder that already holds a database. It is also the first screen on a fresh install, so *About Kinesis* 4 opens a short walkthrough of the app, a page a section. Opened from inside a workspace (Ctrl+K), the walkthrough calls each section by that workspace's name for it. Behind it runs Conway's Game of Life, which a click on the background seeds.

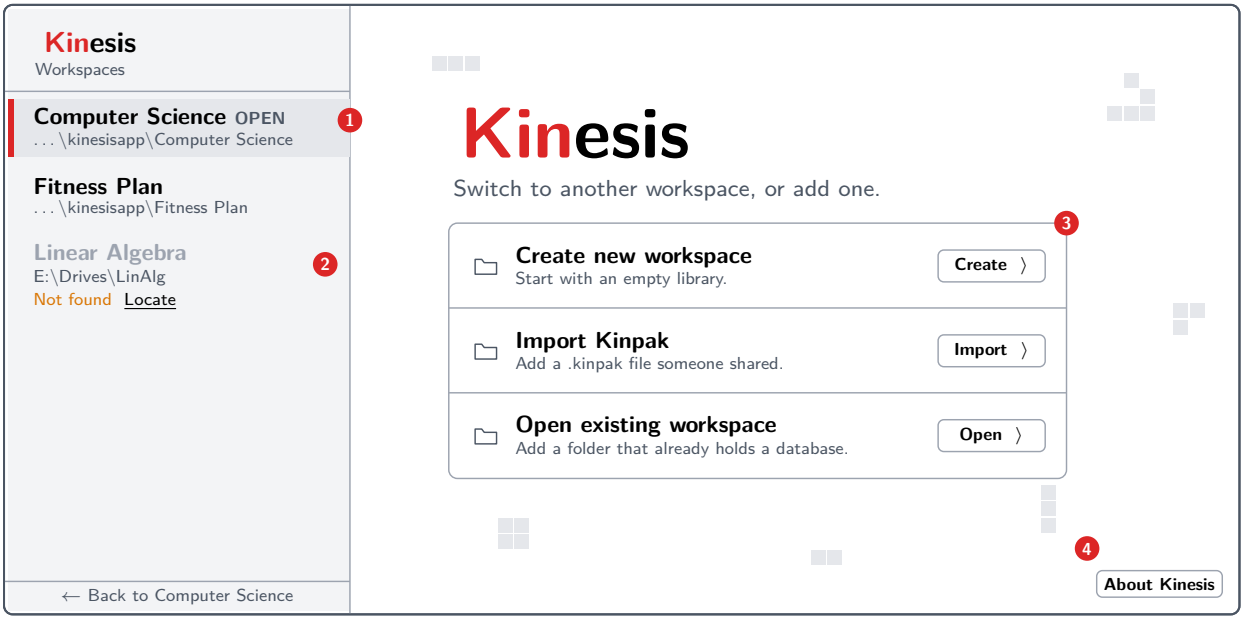


Figure 3: The Workspaces screen, with the open workspace, one whose drive is unplugged, and the ways to add one.

3.2 The database

Each workspace is one SQLite database, accessed through `rusqlite` with SQLite bundled. Three rules shape the schema:

1. Every table is declared `STRICT`. A value of the wrong type is an error instead of being quietly converted. This caught real bugs, such as a view count that came back from YouTube as `"1,234"`.
2. There are no foreign keys. Cleanup happens in triggers: deleting a track deletes its entries and their completions, deleting a video removes it from Drive sequences and pins, and so on. The production database is maintained by hand outside the app, and triggers fit that workflow better than foreign key enforcement.
3. The production schema is checked in as `production_schema.sql`. A test opens a database built from it, runs `init_db` twice and asserts that nothing was added or removed. Any new table, column or trigger has to be added to that file, or the test fails.

New columns arrive through a small migration step: if `column_exists` says a column is missing, an `ALTER TABLE ... ADD COLUMN` adds it. Triggers that depend on new columns are created after that step, since an older database may not have the column yet.

3.3 Main tables

Table	Holds
`Videos`	One row per saved video: title, author, handle, length, transcript, summary, view count, publish date, comma-separated tags, home Drive (`WDBS`), date added, and search tokens.
`ftsVideos`	The FTS5 index over title, summary, tokens and Drive.
`tblWDBS`	The Drive tree: every node, with its alias, icon, color and markers.
`VideoWDBSLinks`	"Also in" links that file a video under more Drives than its home.
`DriveSequence`	Ordered watch lists, one per Drive.
`Glossary`	Quick Tags and Terms. One row per definition, with the Drives it is filed under.
`Biographies`	People and channels: display name, the reader's alias for it, bio, social links, channel ID, subscriber count.
`VideoNotes`, `VideoAttachments`, `AttachmentBlobs`	Notes and files attached to videos.
`Calendar*`	Tracks, entries, completions, in-progress marks, terms, holiday sets and rules (Section 12).
`PinnedVideos`	The reader's pinned videos.
`Notes`, `PersonalNotes`	The Notes section's notes, shared and personal (Section 14).
`Workbooks`, `PersonalWorkbooks`, `WorkbookNotes`	Workbooks, and the notes each holds, in order.
`Logs`, `LogEntries`	The reader's logs, a row a day (Section 15).
`OrphanedVideos`	Saved videos YouTube no longer has, with what YouTube said and when it was found.
`Settings`, `WorkspaceLabels`	Settings and feature flags; the workspace name and section names.
`SearchHistory`, `SyncItems`	Search history; which rows a sync server owns.

3.4 Attachments

Files attached to a video are stored inside the database and addressed by content. `AttachmentBlobs` keeps each distinct file once, keyed by the SHA-256 of its original bytes, and `VideoAttachments` points videos at blobs. The same PDF attached to ten videos is stored once. Compressible types are deflated, but only when the result is smaller and only after decompressing it proves it gives back the original bytes exactly. Every read checks the bytes against the hash. A file must be under 128 MiB. Web links are stored as attachments of a special type whose bytes are the address, so they travel with everything else attachments already do.

4 The Drive system

Videos are filed in a tree called the Drive, written as paths such as `:CS-DSA` (Computer Science, then Data Structures and Algorithms). Internally it is called WDBS, for Workspace Drive-Based Structure. A workspace can rename the section (a course workspace might call it "Modules"), so code and comments say "Drive" and the interface shows whatever name the workspace uses. The Manage Drive window is the exception and always says "Drive".

4.1 Two spellings

A path has a display form and a storage form:

Display form	Storage form
`:CS-DSA`	$\theta\psi$`CS_DSA`

The storage form is what the production database uses in `Videos.WDBS` and `VideoWDBSLinks`. The display form is used in `tblWDBS`, sequences and glossary filing. Conversion happens at the edges, and code that touches both must be careful about which one it holds.

4.2 Homes and links

Every video has one home in the tree, or none ("Unsorted"). It can also be linked into other Drives, which is how a lecture on hash tables appears under both `:CS-DSA` and `:INTERVIEW`. The production database has a trigger that rejects a home path that is not registered in `tblWDBS`, so the app registers a path and every level above it before filing a video there.

4.3 Changing the tree

Renaming, moving or merging a Drive is one operation, `relocate_drive`. A path is referenced from six places, and all six are updated together in one transaction: `tblWDBS`, `Videos.WDBS`, `VideoWDBSLinks`, `DriveSequence`, the Drives a glossary row is filed under, and `kinesis://drive/...` links inside text.

4.4 Sequences

A sequence is an ordered list of videos for one Drive, shown in the sidebar as First, Previous and Next, with *Jump To* beside them: an empty box for a place in the sequence, and Enter opens the video there. What is typed is kept inside the sequence as it is typed, so anything under 1 becomes 1 and anything past the last place becomes the last place. Like Previous and Next, it is off for a video that is not in the sequence. The Library's Drive panel has a toggle in its heading between the tree and a list of every Drive's sequence, each drawn as a playlist; clicking one opens it as a sequence link does (its first video, with the bar following that sequence), and a button on its row shows that Drive's videos in the Library instead. A filter over the list narrows it by name, Drive or first video. The choice of view is remembered. A sequence may include any video filed at or below its Drive, so `:CS` can walk across `:CS-DSA` and `:CS-OS`. The same video can sit in the sequences

of several Drives, in different positions. Positions are kept as 1 to N on every write; reads only depend on their order, so a gap left by a deleted video does no harm.

5 Getting videos in

The Search section finds videos on YouTube through the same internal endpoints the YouTube website uses (`youtubei/v1/search`, `browse` and `player`), so searching needs no API key. Pasting a video, playlist or channel link, or typing a `@handle`, turns into a filter chip automatically.

Transcripts come from the caption tracks the player lists. When YouTube answers with "Sign in to confirm you're not a bot", the app treats it as YouTube limiting the network rather than a problem with the video, because asking again later often works.

A YouTube Data API key is optional. With one, the app fetches the channel ID and subscriber count the first time it sees a channel and stores them on the channel's biography. The channel ID is captured at that moment because, unlike a handle, it cannot be recovered later if it was never saved. A missing subscriber count is stored as -1, which is never a real count.

Saving a video computes its search tokens: the distinct, lowercased, punctuation-stripped words of the transcript that are not stop words. The original SQL for this asks SQLite for the character at each position, which gets slower the further into the text it goes. That is fine for one video and takes minutes for an import of thousands, so imports use a Rust version that reproduces the SQL's rules exactly, quirks included. A test compares the two.

6 Search and filtering

Library search runs on SQLite's FTS5. Each typed word becomes a prefix match. Facets narrow the search and are typed as prefixes that turn into a chip:

Typed	Meaning
`#name`	Videos with this Quick Tag
`^name`	Videos with this Term
`@handle`	Videos from this channel handle
`@@name`	Videos from a channel by display name or alias
`>id`	One video by its ID
`!#`	Videos with no tags at all
`:CS`	Videos in this Drive or below it

A `*` matches any run of characters, so `*harbor` finds "codeharbor". Ghost text suggests the rest of a tag, term, handle, channel name or video ID as it is typed, and Tab accepts it. A channel's alias is suggested after its real name, so `@@` finds a channel under either.

The Biography's search bar has one facet of its own: a Drive path typed after `:`, such as `:CS-DSA`, becomes a chip that keeps the creators with a video filed in that Drive or below it, home or "Also in". Unlike the other chips, it holds its own value, and text typed after it still searches names.

The Similar Videos tab in the sidebar ranks with BM25 over the same index, weighting title above summary above tokens (2.0, 1.5 and 1.0) and giving the Drive column no weight, so that two

videos in the same folder are not called similar for that reason alone.

The Calendar's search bar is a filter rather than a search. Plain words match an item's name, description or track, and one facet at a time (`track:`, `importance:` for an item's priority, `status:`, `kind:`) narrows by that field. Cells holding a match are tinted instead of hiding everything else, so the week does not jump around while typing. Enter moves to the next matching day, up to a year away.

7 The sidebar

Opening a video opens the sidebar over whatever section is showing (Figure 4). It holds the player, the transcript and the AI summary (both editable as Markdown), the video's Drive home and links, and under the player a row of tabs: Terms, Tags, Attachments (the video's note and its files), Similar Videos and Backlinks. When the sidebar is too narrow for all of them, the tabs that do not fit move, from the end, into a *More* dropdown rather than having their names cut short. The header has a pin button beside the close button ❶.

Under the header, a strip shows the Calendar tracks the video is on ❷: which track, its place in the list, its day, an *In progress* toggle and *Mark done*. When the video plays to its end, the embed page says so and a window asks whether to mark it done on those tracks. It asks rather than marking the video, because people rewatch videos they have already finished.

The embed page also reports where the video is every few seconds while it plays, and on a pause. The app keeps the latest position per saved video in `VideoPositions`, and opening the video again offers *Resume from* that moment on the player. It is an offer rather than a jump, it is not made near either end of the video or when the video was opened from a timed link, and playing to the end forgets the position. Watching past the first half minute marks the video In progress on its tracks. The pin menu lists started videos as *Continue Watching*, and finished ones (played to the end, or marked completed there, which also checks them off on their tracks) as *Watched*, kept in `VideoWatched`. Like pins, positions and the watched list are the reader's own: they go through the Trash with their video and are never put in a Kinpak.

The right-hand pane ❸ switches between the transcript and the summary. Timestamps typed in square brackets, such as `[1:23]` ❹, become links that jump the player. The brackets are required so that a ratio like 2:1 or a clock time is not mistaken for one. A summary ends with a "Channel Info" footer ❺ that links to the creator's biography; when the reader has given the channel an alias, the alias shows as a badge in front of it. Editing guards against losing work: leaving the sidebar, switching videos or quitting with an unsaved edit asks first.

8 Keeping up with YouTube

A saved video is a snapshot. Channels rename themselves, change their handles, retitle videos and take videos down, and none of that reaches the library by itself. Kinesis checks the things a reader is looking at, through the same keyless endpoints it uses everywhere else.

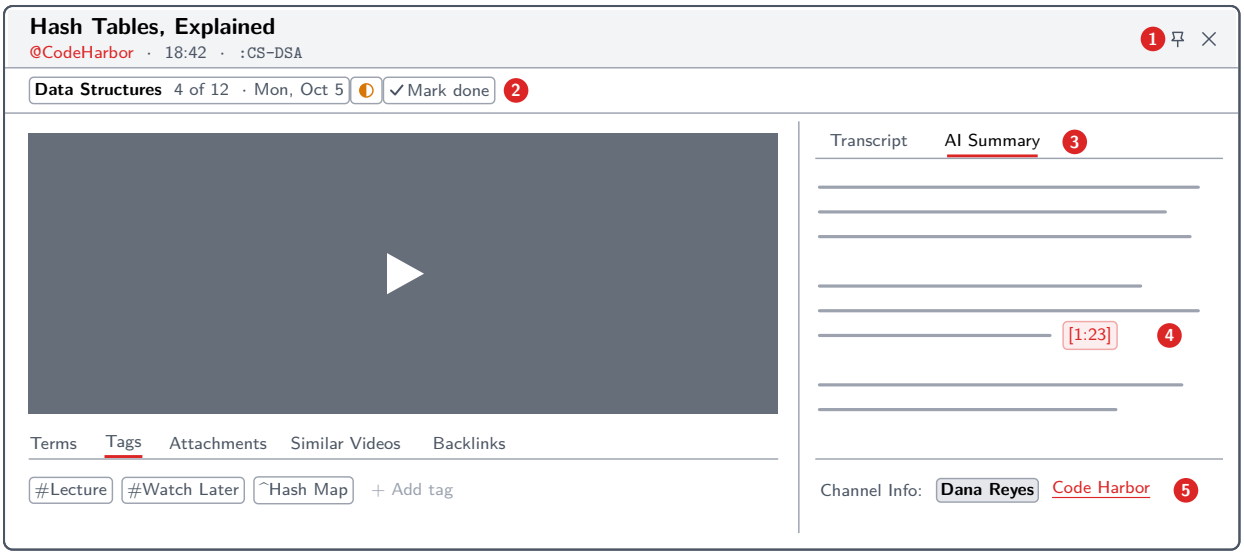


Figure 4: The sidebar. The track strip only appears when the video is on a Calendar track.

8.1 Renamed channels

Opening a biography looks the channel up by its channel ID, which never changes, on the `browse` endpoint. The answer's `channelMetadataRenderer` gives the channel's current name and, from its vanity URL, its handle. The answer must carry the same channel ID, so a redirect to some other page is never taken for a rename. A biography saved before channel IDs were captured gets its ID from its handle the first time this runs.

A new handle is taken over at once: the biography's row, its videos, its custom prompt, every `kinesis://bio/` link in any text, and the Channel Info footers of its videos. If another biography already has the new handle, nothing changes, since merging two people is not something to do without asking. A new display name is different, because the reader may know the channel by its old name. The window asks whether to keep the old name as the channel's alias (Section 10.1); either way the biography takes the new name. In a read-only workspace both are applied without asking, and no alias is made. A workspace can also be set not to ask (`doNotAskToAlias`, *Do Not Ask to Alias* under Permissions): the new name is taken and the reader is only told, and an alias the channel already has is left as it is.

A stored name sometimes includes the handle, as in "Code Harbor (@CodeHarbor)", because that is how some names arrive with a video. The comparison ignores a trailing handle in brackets, so such a name does not count as a rename.

8.2 Renamed and removed videos

Videos are checked on the `player` endpoint (Figure 5). Outside a browser YouTube refuses to play a video, but it still returns the title when the video exists. A removed video answers `ERROR` with no video details at all. That gives three outcomes:

- A title came back. The video exists; if the title changed, the library takes the new one. A video opened in the sidebar also shows a short *Video Title Renamed* notice with an OK button. There is nothing to choose, since a video has no alias.

- `ERROR`, or a private video. The video is marked orphaned. Its title is left alone, because there is no new one to take.
- Anything else, including "Sign in to confirm you're not a bot", an age gate, a region block or no answer at all. Nothing is concluded. A sign-in check is about the network asking, not about the video, and orphaning a video over it would be wrong.

Two things start a check. Opening a saved video in the sidebar checks it once per opening. The Library's thumbnails are the other: YouTube serves a removed video's thumbnail as a 404 with a 120 by 90 gray placeholder, where a real one is 480 by 360. A card whose thumbnail fails or comes back as the placeholder queues its video for a check, and so does a good thumbnail on a video that is currently orphaned, since the video may have come back. The thumbnail only starts the check; it never decides it, so a network hiccup cannot orphan a video. The queue runs one video at a time with a short pause between them and asks about each video at most once a session, so scrolling past a page of broken thumbnails does not flood YouTube.

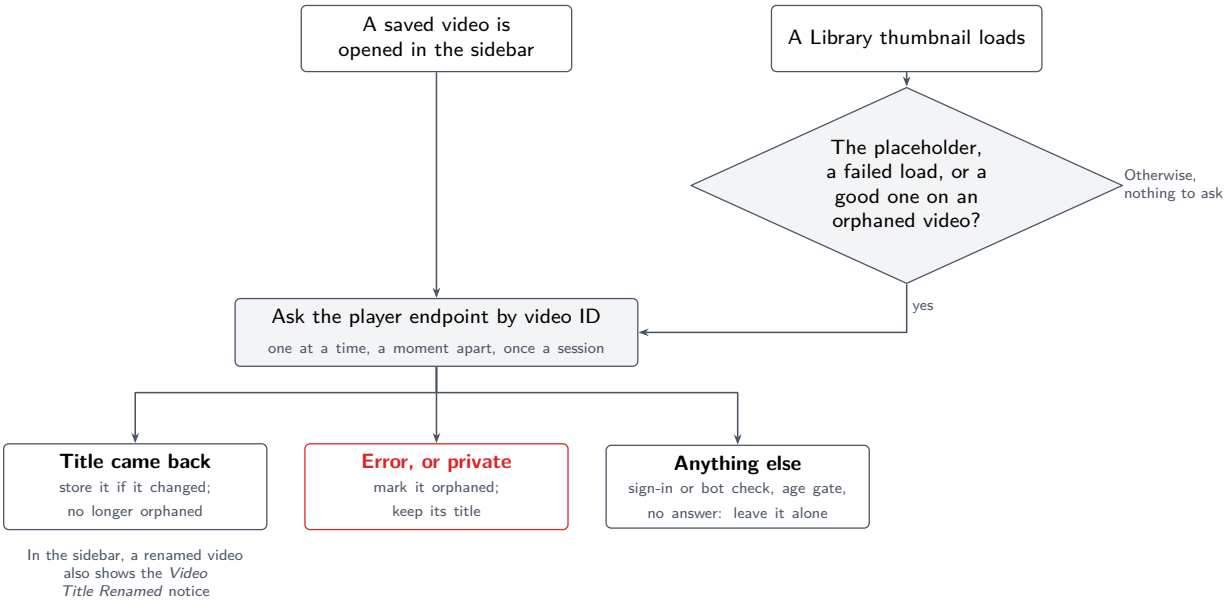


Figure 5: Checking a saved video against YouTube.

8.3 Orphaned videos

An orphaned video stays in the library, because its transcript, summary, notes and links are still worth having. The `OrphanedVideos` table records it, and the Library's status bar shows an *Orphaned* chip next to the Trash (Figure 2, 6). The chip opens a list of orphaned videos, each of which can be opened or deleted, with a *Delete All*. Deleting goes through the Trash like any other delete. A video that YouTube answers for again is taken off the list.

Delete All is one bulk delete rather than a loop of single ones. The rows go in one transaction, and the links to all of the videos are removed in one pass over the stored texts, with each video's own text changes recorded separately. Every video still becomes its own Trash item and can be restored by itself.

9 Links between things

Text anywhere in Kinesis can link to other things with Markdown links on a `kinesis://` address:

Address	Opens
`kinesis://video/ID?t=1:23`	A video, optionally at a moment
`kinesis://glossary/Term`	A glossary term
`kinesis://bio/handle`	A person's biography
`kinesis://drive/...`	A Drive in the Library
`kinesis://playlist/:CS-DSA`	Whatever is first in that Drive's sequence

Links can live in six places: video summaries, transcripts and notes, biographies, glossary definitions, and the notes of the Notes section (Section 14). In a Markdown editor, typing `@` or `^` followed by the start of a handle or term shows the rest as ghost text, and Enter or Tab turns it into a link.

Renaming or deleting a term, person or video rewrites every link to it. The scan looks only at texts containing `kinesis://`, which takes about a tenth of a second for the summaries of five thousand videos. Backlinks use the same scan in reverse. A summary's "Channel Info" footer links to the creator, but backlinks and the graph view ignore that footer: every summary has one, so a person's backlinks would otherwise just list all of their videos again.

10 Glossary and Biography

The Glossary holds two kinds of entries. A Quick Tag is a label with no definition. A Term has a definition, written in Markdown. A term can have a different definition in different Drives, so the table stores one row per definition with the list of Drives it is filed under. A term can be filed at any level, such as FRENCH or just FRENCH-CH1. Filtering the Glossary by a Drive shows the terms of the Drives beneath it too, and a video in FRENCH-CH2 is offered FRENCH's terms as well as its own; where a term has definitions at both levels, the closer Drive's wins.

Five Quick Tags are built in: Watch Later, Favorite, Revisit, Key Source and Follow Up. They cannot be renamed or deleted, and the app puts back any that are missing each time a workspace opens.

The Glossary can turn the terms on screen into flashcards (Figure 6): all of them, or one Drive's. A card has the term on one side and the definition on the other, and *First* ❶ picks which side faces up. *Shuffle* mixes the deck and *Start over* returns to the first card ❷. Clicking the card, or pressing Space, turns it over with a 3D flip ❸; a definition keeps its links, so a term can lead straight to the video it came from. The arrows, or the arrow keys, move between cards ❹.

10.1 Biographies

A biography belongs to a channel handle. The app creates one the first time it saves a video from that channel, filling in the display name, and never overwrites a display name the user has set. Biographies carry a Markdown bio, social links and backlinks.

A biography can also have an alias: a name of the reader's own for the channel, usually the person behind it. Channel names are often brand names, and "Code Harbor" says less to most

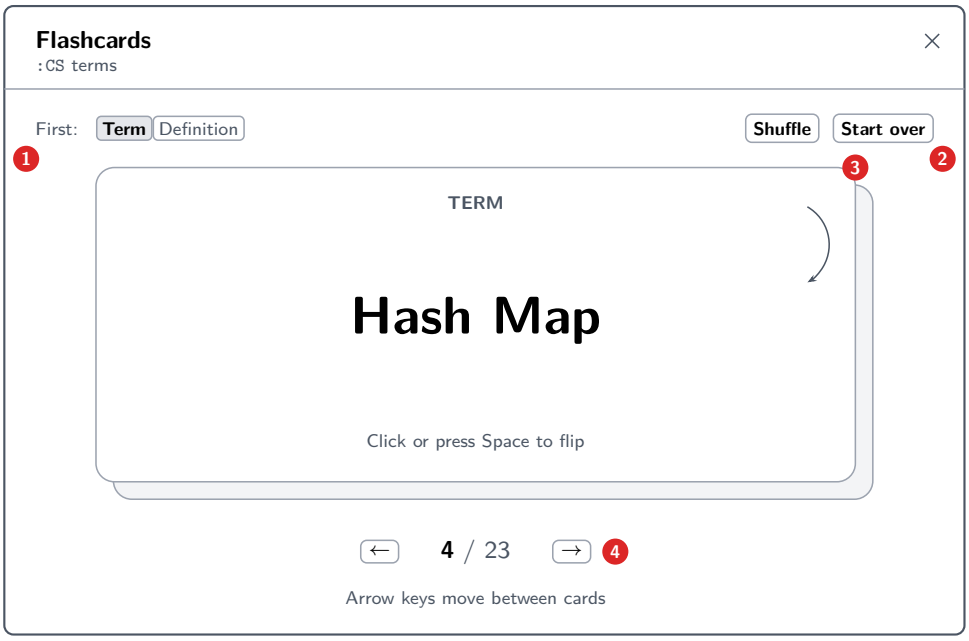


Figure 6: Flashcards for one Drive's terms, showing the term side.

readers than "Dana Reyes", the person who hosts it. The alias is optional and is set in Edit Bio. Where it is set, it shows as a badge in front of the channel's name (Figure 7, 1), in the Biography list, in the biography's window and in a summary's Channel Info footer. `@@` searches match it as well as the name, and the Obsidian export writes it as the note's `aliases`, so a wiki link to the alias finds the note.

Edit Bio has the alias and the handle on two tabs, Alias and Handle Casing. The handle can only change its capitals, so `@john` can become `@John` but not `@jean`: the handle is the channel's identity on YouTube, and a typo there would point the biography at someone else. The window refuses anything else before saving, and the backend checks again. A change of capitals is carried through like a rename from YouTube (Section 8), so videos, links and footers follow.

Show Alias, a switch in the Biography status bar 2, is on by default. On, the badges show and the list is sorted by alias where there is one; off, the badges are hidden and the list goes by the channels' own names. The biography's own window always shows the badge. The setting is the reader's, kept in the browser's local storage. *Alias Only*, beside it, shows a channel with an alias by the alias alone, in plain type like any other name, and marks its row with a small italic *a* on a tile in place of the bullet (hovering it says *Alias*).

When a channel has renamed itself on YouTube, opening its biography asks whether to keep the old name as the alias 3 (Section 8). That is the case the alias is most useful for: the reader still knows the channel by the name it used to have.

11 AI summaries and images

Summaries are made with Venice. A workspace can store custom prompts per channel. After a summary is saved, the app appends a "Channel Info" footer naming the channel and linking it to its biography when one exists.

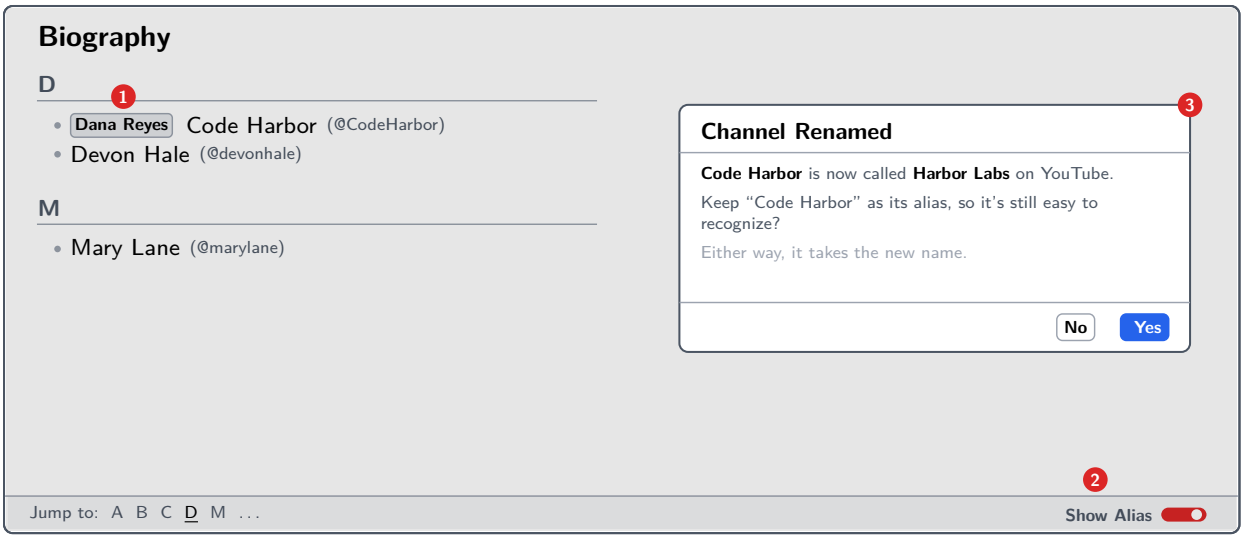


Figure 7: The Biography list with an alias badge, and the prompt shown for a channel that renamed itself.

Summaries can also carry images, generated by Venice, taken from Pixabay, or uploaded. All of these features are optional and can be hidden by the workspace owner.

12 The Calendar

12.1 Tracks

A track is an ordered list of entries. An entry is a video, a note (Section 14) or a task; a task has a name, an icon, a description and a priority from none to high. A video entry has the same description and priority, edited in the same window, but keeps the video's title and icon. Each track advances in one of three ways:

- *One after another.* The first entry not yet done is up next. There are no dates.
- *On a schedule.* The track has a start date and a pattern of days (below), and the nth scheduled day belongs to the nth entry, whether or not it was done.
- *Schedule and finish.* Entries get days as above, but the track waits for each one to be done. An entry past its day and not done is overdue.

A track can loop, which makes it a rotation. Each time through the list is a cycle, and completions are stored per entry per cycle. The data model also lets a dated track be limited by terms (a semester, or a range that repeats every year) and skip or include holiday sets, with US federal holidays calculated rather than stored. These are used by the schedule, by imports and by the ICS export, but the app has no screen for editing them.

12.2 Patterns

A dated track's days follow a pattern, counted from its start date. There are three kinds, and a track has one:

Kind	What it means	Examples
By week	Its weekdays, every n weeks	Mon/Wed/Fri; alternate Tuesdays
Every n days	Every nth day from its start	Every other day; every 14 days
By month	Its weekdays, first to fourth or last	The first Monday of the month

Repeats, in the track window, is a dropdown of the usual ones by name, with *Custom* for the rule itself: Every day, Weekdays, Mon/Wed/Fri, Tue/Thu, Every other day, Every other week and First Monday of the month. Custom is what it shows whenever the pattern is none of them, and it opens the rule beside it: Weekly every n weeks, Daily every n days, or Monthly on the first to the last. "Every other week" keeps whichever days are picked and runs them every second week, counting whole weeks from the week the track starts in, so two tracks can take alternate weeks by starting a week apart. The days of the week are only asked for where there are days to pick: a pattern named for its days already says them, and a daily count lands wherever it does, weekends included. Under the schedule, the window says the pattern back with its first four dates, because which week an every-other-week track begins on is easy to get wrong and hard to see otherwise.

Terms and holiday sets apply on top of the pattern. A day the pattern lands on that a holiday set takes out is simply not a day on the schedule: no entry is used up, and the pattern keeps its rhythm rather than shifting. A reader who wants the day off to show can add a task named for it instead.

The pattern is stored on the track itself, in three columns: its kind, its n, and which of the month's weekdays (`repeat_kind`, `repeat_every` and `repeat_ordinal`). Only what a kind uses is kept, so two tracks that repeat the same way are stored the same. A track from before patterns is its weekdays every week, which is what it always was.

An entry can also be a folder: a named group of videos and tasks that share one place in the track. The schedule deals out slots rather than entries, where a slot is an entry on its own or a folder with everything in it, so a dated track gives a whole folder one day and several things land on the same day. An undated track keeps a folder up until everything in it is done. A folder is only a name; what it holds keeps its own priority and description. Folders are one level deep. Whether a folder is done or in progress is worked out from what it holds, never stored: done when all of it is, in progress while some of it is started or done. Checking a folder off checks off everything in it, an empty folder still holds its day, and removing a folder leaves its entries in the folder's place. In the tracks panel, entries are dragged into a folder by dropping them on the lower half of its row, or next to an entry already inside it.

A track can also follow a Drive's sequence. Its entries then come live from the sequence, and only entries with something of their own (a completion, a description) have a row.

The state of a track (which entry is up, how far along it is, what is overdue) is never stored. `resolve` works it out from the entries, completions and schedule each time it is asked. This keeps the stored data small and means there is nothing to fall out of date.

Items can be checked off in three places: the Calendar, the sidebar's track chips, and the tray popup's Today tab. The pin's due-today count reflects all of them. Every call in `api.ts` that changes the Calendar announces it afterwards, as an app event that reaches every window, and

whatever shows Calendar data reloads when it hears it. A check-off in the tray popup therefore shows in the main window straight away.

12.3 The week

Figure 8 shows the Calendar. The tracks panel on the left lists the tracks and, under *Up next* 1, what is next on undated tracks and on dated tracks that have fallen behind. The week has a row per track and a column per day, with each entry on its day as a chip. A chip carries three kinds of mark:

- Done 2: a check, with the name struck through.
- In progress 4: a circle with its left half filled, in amber. It means started but not finished, and is set from the item's menu, the track panel, or the sidebar's track strip. It is stored per entry per cycle in `CalendarProgress`, and marking the entry done clears it. Overdue is not a separate state but a flag on top: an item can be in progress and overdue, and the `status:` facet finds it under both.
- Priority: a diamond, a gray outline for low, half filled yellow for medium, solid red for high. The fill grows with the level, so the levels can be told apart without seeing the colors.

While the search bar has a filter, cells holding a match are tinted 3 and the rest of the week stays in place. Clicking a chip opens its menu: Mark Done, Mark In Progress, Open Video, and for a video Pin or Unpin; then, where the workspace can be edited, Change Priority (a second level: None, Low, Medium, High), Open Description when there is one, and Edit Task (or Edit Video, Edit Note), which opens the same window the track's panel does.

The tracks panel's header has two checkboxes: *Show Calendar*, on by default, which hides the small month over the list when the reader only wants the tracks, and *Archived*. A track's entries are added from under its list, under an *Add* heading: Video, Note, Task, Folder. Looking up a video lists the pinned ones first, under *Pinned*, then *All Videos* without them again, since what is pinned is most likely what is wanted.

The track window is in three parts: the track's name and how it moves along; its *Schedule*, for a track that follows dates, a row to a setting (Repeats, Days, Starts, Time); and its *Options* (looping, its Drive). Hovering *Start over after the last one* says what on and off each do. The window asks before closing with changes unsaved. *More Info*, beside *How it moves along*, says what each of the three ways does and what it suits, and changing the way an existing track moves along shows a warning first, since it changes what is up next and which days its items fall on. On Linux the webview's date picker stays open after a day is picked, so the app's date fields (`ui/DateInput`) let go of focus there once a pick lands.

The left column's panels fill it down to *New Track*: the list of tracks, and an open track's panel, whose heading stays at the top and whose *Add* row stays at the foot while its items scroll between them. The Logs list and the Library's Drive and Sequences panels fill their columns the same way.

12.4 Time windows

A dated track can have a time of day, such as 9:00 for 90 minutes. Two tracks clash when their times overlap and there is a day both their patterns land on while both are running. The days are worked out rather than read off the weekdays: the two patterns are compared day by day for two

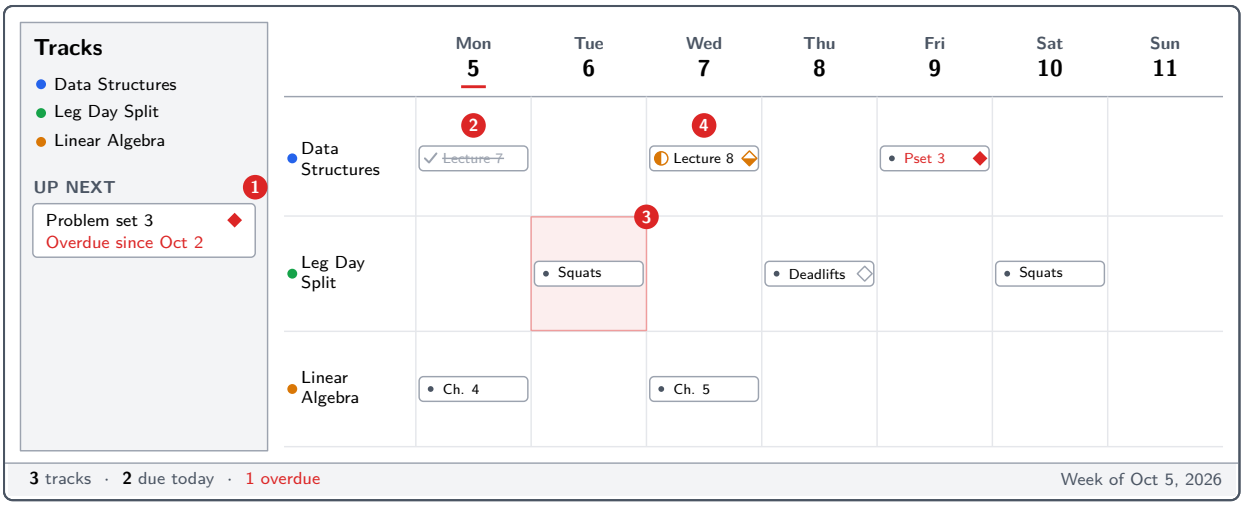


Figure 8: The Calendar's week, with a done item, an item in progress, three priorities and a tinted search match.

years from the later start, which is past where any two patterns come round together. So every third day and alternate Mondays clash (they meet within six weeks, and the message names the day), while two every-other-week tracks on opposite weeks never do. A track runs from its start date to the end of its last term, or forever if it has no terms or has a term that repeats yearly. The check runs inside the same transaction as the change that could cause a clash (creating, editing or unarchiving a track, or changing its terms), so a clash rolls the change back. Archived tracks are never checked.

Track names are unique among active tracks, ignoring case and repeated spaces. An archived track can share a name, so last year's "Data Structures" can sit beside this year's.

12.5 Starting over

Start over sends a track back to its first entry in round 1, from today. Nothing is deleted. The track's `cycle_offset` moves past every cycle used so far, so old completions remain as history under their own cycles and the new run starts clean. A dated track's start date moves to today.

The previous start date and offset are saved, and the start over can be undone until something is marked done in the new run. Undoing after that would leave completions in a cycle the old run has not reached. Editing the start date by hand also clears the saved state, since undoing would then restore a date from before the edit.

12.6 Deleted videos

When a video is deleted, its entries stay in their tracks as tasks named after the video. The entry remembers the video's ID in `former_video_id`, and a trigger on inserting into `Videos` reconnects it if the video comes back, through a Trash restore or by saving it again. A name the user typed by hand is kept through all of this.

12.7 Export

Dated tracks can be exported as an `.ics` file for Google Calendar, Apple Calendar or Outlook, for any period up to two years. The writer follows RFC 5545: text is escaped, lines are folded at 75 octets and end in CRLF. Every event has a stable UID made from the track, entry and cycle, so importing a newer export updates events instead of duplicating them. Tracks with a time window export timed events in floating local time; the rest export all-day events.

13 Pins

A pin marks a video the reader is working on now. The sidebar's pin button toggles it, and the pin menu in the title bar lists pinned videos, newest first, from any section. Under the list, a row shows how many Calendar items are due today, and a second row, shown only when tomorrow has anything, how many are to do tomorrow. Each opens the Calendar on that day's week with its items flashing briefly. Only today's count appears beside the pin icon; tomorrow's is a look ahead and is not added to it. Then *Log Entry* opens a submenu of the reader's logs. Picking one turns the submenu into that log's row for today, an Entry and a Description to type in (showing what today already has), so a day is logged from wherever the reader is, a video included, without leaving it. Ctrl+Enter or *Save* writes the day as the sheet would, and *Open log* goes to the log itself. The form stays while it is being typed in, whatever row of the menu the pointer passes over. A log with its bell on and nothing written today (Section 15) is marked there with a bell, the number of them is shown beside *Log Entry*, and they are added to the count beside the pin icon, whose tooltip says how much of it is the Calendar's and how much the logs'. Below that, each Essential tag opens a submenu of the videos carrying it. A video can also be pinned from its chip on the Calendar.

Figure 9 shows the menu open. The count beside the pin ❶ is what is due today. Each pinned video ❷ can be unpinned from the menu with the button that appears on hover. The Calendar rows ❸ sit under the list, and the Essential tags ❹ open their submenus to the left ❺, because the menu sits at the window's right edge. A submenu ends with *Show all in Library*, which opens the Library filtered to that tag.

Pins have their own table rather than being a tag. A tag would travel in a Kinpak, would carry no time for ordering, and would show up as a chip on every pinned video. Pins are meant to stay a short list; there is no need to search for them.

14 Notes

A note is a page of the reader's own writing. It does not belong to a video: it is study notes on a topic, a chapter or an exam, and it *cites* videos the way an essay cites sources. A citation can be a whole video, a moment in it, or a passage of its transcript or summary. This is different from the note a video already has (at the top of the sidebar's Attachments tab, stored in `VideoNotes`), which is one text about that one video and its files.

Notes were first considered as comments pinned to passages of a transcript or summary, made by right-clicking the text. That was set aside for two reasons. Summaries are regenerated and transcripts are edited, so a comment pinned to a stretch of text needs that text found again after every change, and some comments would inevitably be orphaned. And studying is mostly gathering: what a chapter says, drawn from several videos, in one place. Comments scattered across twenty

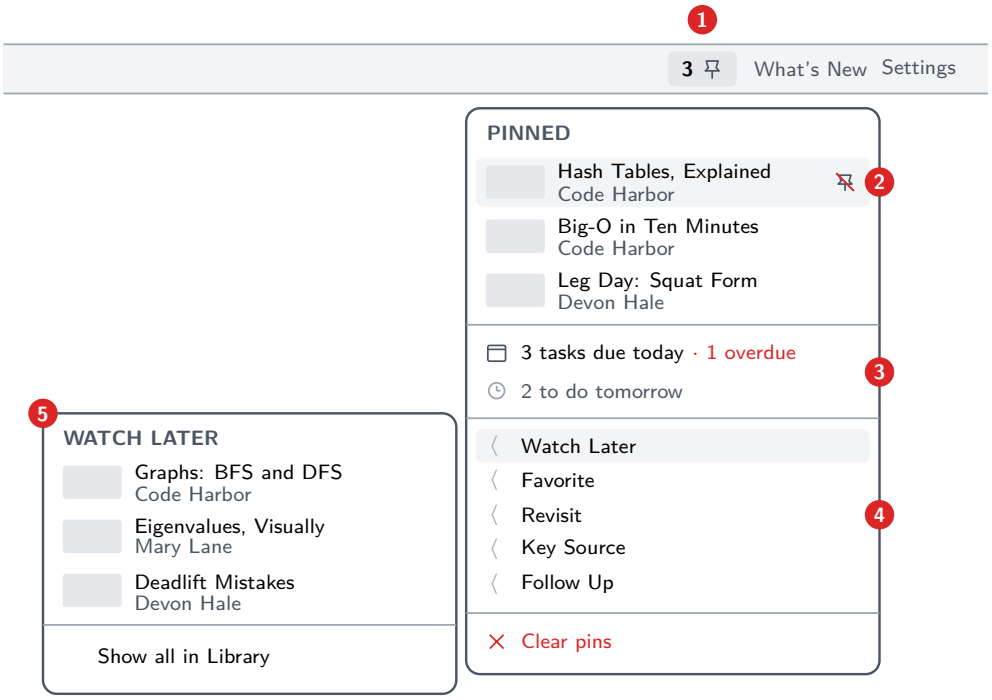


Figure 9: The pin menu with the Watch Later submenu open.

transcripts do not add up to that.

14.1 The page

Notes are a main section, after the Calendar (Figure 10, 1), laid out like the Library and the Calendar: the search bar on top, with a button beside it that shows and hides the list 9, then the list of notes, the open note and what it cites, each scrolling on its own. The search bar filters the list as it is typed in: plain words are looked for in titles and text, and `title_search:` (typed as `!n`) in titles only. *New Note* 2, at the foot of the list as *New Track* is in the Calendar, gives a blank page titled *Untitled* 3; typing in the title names it, and a note left unnamed is listed by its first line. A new note is personal, and is filed under the Drive the list is filtered to, if any. There is no Save button: a note saves itself shortly after typing stops, and saves what is waiting when the section or the app closes.

The arrow beside *New Note* opens the templates a note can start from instead of a blank page: a few that come with the app (Lecture Notes, Weekly Review, Summary, Question Sheet), then the workspace's own, which are made from a note with *Save This Note as a Template* and kept as one JSON list in the workspace's settings (`noteTemplates`). A template is a name and the title and text a note starts with; `{{date}}` in either is written out as the day the note is made. Making a note copies the template, so changing one later changes nothing already written.

Deleting a note asks first, whatever *Confirm Before Deleting* is set to, and sends it to the Trash: a note is someone's writing. The exception is a note with nothing of theirs in it, one that is empty or still exactly what its template gave it. That one is deleted without asking and for good, with no place in the Trash and no Undo.

Write shows the Markdown being written, in the same editor as summaries and definitions,

so the formatting shortcuts, `@` and `^` links with ghost text and the link picker (Ctrl+Shift+K, which has a Notes tab) all work. *Read* shows the note as it reads, with citations as cards. The choice between the two is the reader's, not the note's: it carries from one note to the next and is remembered between sessions, so a reader who left reading comes back reading. Starting a new note switches to Write, and a note that cannot be edited shows as Read without changing the choice. The page's tools stay on one line at the right of the note's Drives and never drop to a line of their own: on a narrow page they are drawn as their icons alone, and the Drives wrap in the room that is left. Under the title sit the note's Drives, *Add to Calendar*, and whether it is personal or shared 4. A note is filed under Drives the way a glossary term is, at any level, so a note on GERMAN-CH2 shows when the list is filtered to GERMAN.

The list on the left shows the most recently changed note first, with its first line and when it changed. One figure marks a personal note and two a shared one. Above it is a Drive filter, the same expandable Drive list the Glossary uses. With the list hidden, *New Note* moves up beside the page's other buttons.

Right-clicking a note in the list does, without opening it, what its page can: *Add to Workbook* (a submenu of the workbooks, each ticked when the note is in it), *Add to Calendar* (a submenu of the tracks that can take a note), *Share* or *Make Personal*, *Duplicate*, *Save as a Template* and *Delete Note*. A duplicate is personal whatever the original was, since sharing is chosen note by note, and a template saved from the menu is named after the note, with a number after it rather than replacing one of that name.

A citation 5 shows as a card: the quoted words, then the video's title and the moment. Clicking it opens the video in the sidebar at that moment. A moment is drawn one way wherever a note shows one (a citation, a video on its line, the Sources column): a play mark and the time as a clock, on the accent's tint. A link's time can be written as a clock (`?t=1:27`), as seconds (`?t=87`), or as a timestamp straight after the link (`[1:27]`); all three read as the same moment, and the last is written into the link once the writing is left. A video linked on a line of its own, which other texts unfold into a card with its thumbnail, is one line in a note: the video's title, who made it, and the moment when the link has one. A note is writing, and a row of pictures would bury it. Picking a video in the link picker from a note asks first whether to link the whole video or a moment in it, typed as a time or taken from where the reader stopped watching. The link picker lists the pinned videos first, under *Pinned*, then *All Videos* without them again, as adding a video to a track does.

A `Q:` line followed by an `A:` line 6 is a question and its answer. Read draws the pair as a block: the question, and under it the answer covered until *Show Answer* is clicked, so the question can be tried first. The answer runs on until a blank line or the next `Q:`, and a question with no answer is left as the text it is. The lines can simply be typed; *Question & Answer* in the editor's right-click menu writes them, turning selected text into one (its first line the question, the rest the answer). Nothing is made from these blocks: flashcards are the Glossary's, from its terms.

The column on the right 7, on wide windows, lists what the note points at: the videos it cites with each cited moment, the glossary terms and notes it links, and the texts that link to it (its backlinks). It is worked out from the text each time, never stored. A window too narrow for the column has *Sources* in the status bar instead, which brings the same up as a panel over the page's right edge. The status bar 8 counts the notes listed, and for the open note its words and citations.

Three more things are written in a note. A task's box (`- [ ]`) can be ticked while reading: the line is rewritten to `- [x]`, so Write shows the same. A quote that starts with `[!NOTE]`, `[!TIP]`, `[!IMPORTANT]`, `[!WARNING]` or `[!CAUTION]` is drawn as a callout in that kind's color, with its name as a title; this is GitHub's convention, it works in every Markdown view and not only in notes,

and the editor's right-click menu writes it (pointing at *Quote* opens a panel of the kinds beside the menu). And a link to a Drive's flashcards (`kinesis://flashcards/<drive>`, or `*` for every term), picked from the link picker's Flashcards tab, shows in a note as a small box with a stack-of-cards mark and opens the Glossary's Flashcards on that Drive's terms. It names a Drive, so it follows the Drive through a rename or a merge.

Make Glossary Term, on a selection in a note, makes a term without leaving the note: a few words on one line are taken as the term's name (and become a link to it once it is saved), anything longer as its definition. The term is filed under the note's Drives.

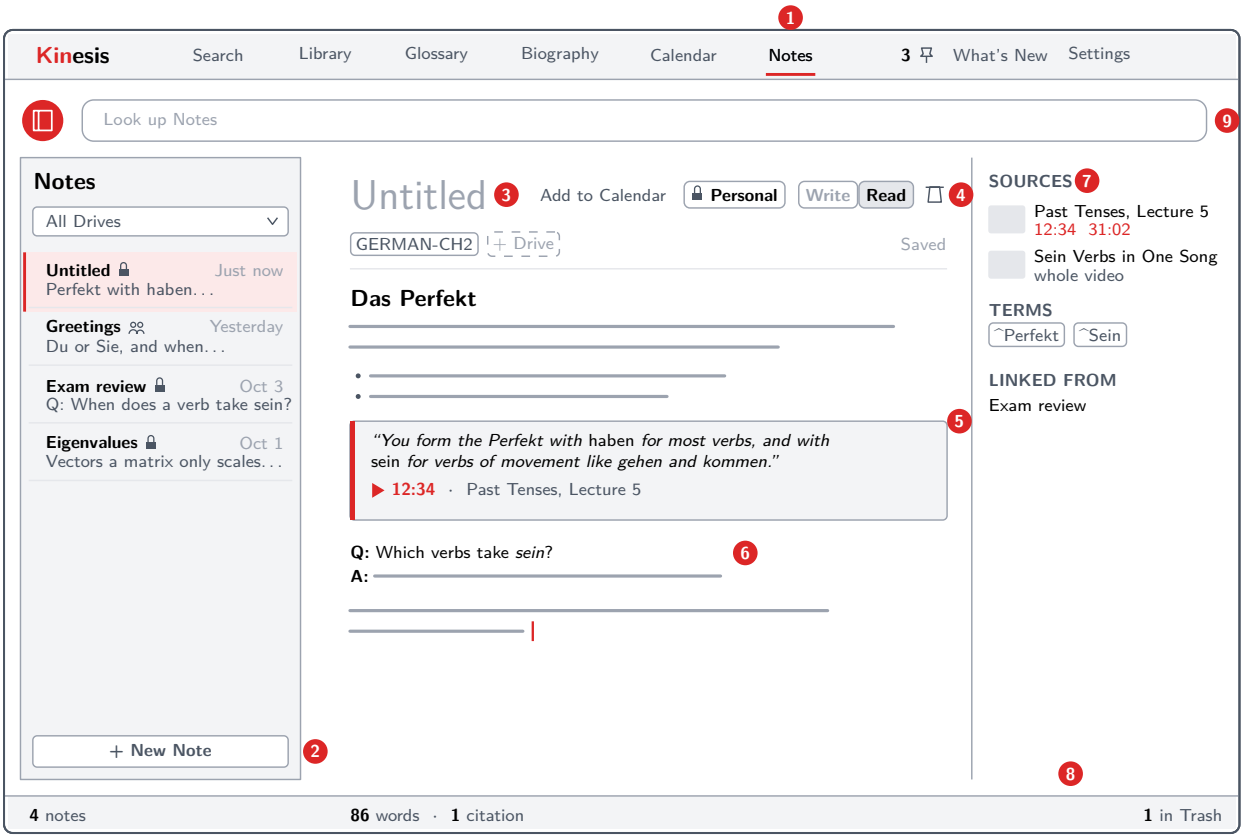


Figure 10: The Notes section with a new note open. It cites a lecture and holds one question and answer.

14.2 Citing

Citing is done from the video, while watching (Figure 11). Selecting words in the transcript or the summary and right-clicking 1 offers, after the two copy commands:

- *Cite in "Das Perfekt"* 2, named for the *active note*, adds them to its end in one step. The active note is the one last opened in the Notes section or cited into; it is remembered in this browser, as a reader's convenience. The item is left out when there is no active note, or it is a shared note this reader cannot edit.
- *Cite in Note...* asks which note, by name or by what it says.

- *Cite in New Note* starts a personal note filed under the video's Drives, with the citation as its text.

Cite Moment 3, beside *Copy Link at Time* on the player, adds a link to where the player is to the active note (or a new one), without pausing. It is the button for taking notes during a lecture: press it, then write the point underneath. A message confirms each citation, with *Open* to go to the note. If the Notes section has that note open, the citation goes straight into what is being written there, so it never races a save. The video's *Backlinks* tab lists the notes that cite it 4.

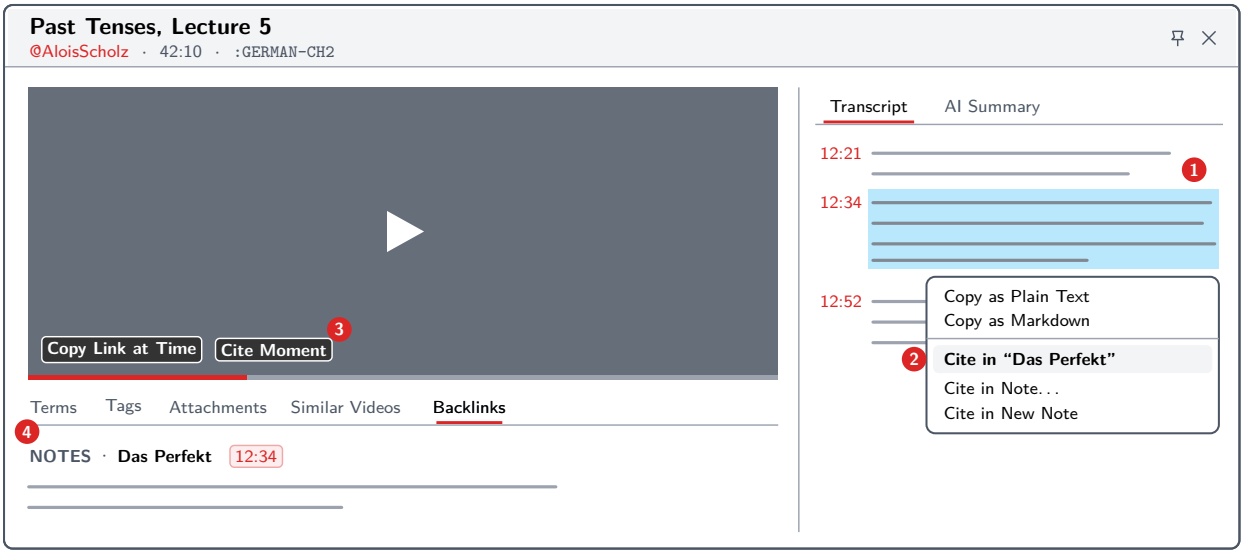


Figure 11: Citing a passage of a transcript into the active note, from its right-click menu.

A citation is written into the note as plain Markdown: a quote, then a line with a link to its source. A transcript's quote is cited at the last timestamp before it, and its own timestamps are taken out of the words; a summary's has no time.

```
> You form the Perfekt with haben for most verbs, and with sein for verbs of movement.
>
> --[Past Tenses, Lecture 5](kinesis://video/abc123?t=12:34)
```

Read shows a quote whose last line is a dash and a video link as a card. Nothing else is new: the link is the ordinary video link, so renaming or deleting the video updates or unlinks it like any other, and backlinks find it. A moment from *Cite Moment* is written as a play mark and a link, `[12:34 · Title](...)`: the mark keeps a video link alone on its line from unfolding into a video card.

The quote is a copy. Editing the transcript or making a new summary never changes or breaks a citation, and the time still opens the right moment. This is the main reason for citing rather than pinning comments to the text.

14.3 On the Calendar

A note can be an entry on a Calendar track, as a video can: a reading, a review or a write-up as a step of its own. *Add to Calendar* under a note's title picks the track, and a track's panel has *Add note* beside *Add video*. The entry is named after the note and follows its title; its name opens the

note, with the note's text as the hover preview, and a chip's menu on the week has *Open Note*. A track holds a note once, and a track that follows a Drive's sequence holds only videos; wherever a note is offered tracks to go onto, those tracks and archived ones are left out, and adding a note a track already holds changes nothing and says so.

The entry stores the note's id (`CalendarEntries.note_id`). Deleting the note leaves the entry as a task named after it, remembering the note in `former_note_id`, and the note coming back (from the Trash) takes its place again, through triggers like the ones that do the same for videos. Moving a note between personal and shared removes and adds its row, so the same triggers keep its places.

14.4 Workbooks

A workbook is notes put in an order of the author's choosing, to be read through a page at a time: a course's notes, chapter by chapter. It holds no text of its own, only a name and that order, so a note can be in several workbooks or in none, and deleting a workbook never deletes its notes. The Notes panel has two sides, Notes and Workbooks. With a workbook open the panel lists its notes, numbered; they are dragged into order by a grip, and a note is put into workbooks from *Add to Workbook* on its page. While the open note is one of the open workbook's, a bar at the foot of the page (outside its scroll, so always there) has Previous and Next, the page number with a box to go to any page, and the workbook's name. The search bar narrows the workbooks, or the open one's notes.

Workbooks are personal or shared, in two tables as notes are (`PersonalWorkbooks`, `Workbooks`), with one table, `WorkbookNotes`, holding every workbook's notes in order. A shared workbook holds shared notes only: what is shared has to be whole for whoever it is shared with. So adding a personal note to one is refused, saying to share the note first, and the workbook's own Personal/ Shared button asks before changing the workbook and its notes together. That change is one transaction in the database: the workbook and every note in it, or none of it. Nothing follows a note being deleted or made personal: its row stays and is simply not listed, so a note restored from the Trash, or shared again, is back in its place.

A workbook is linked as `kinesis://workbook/<id>`, and can be added to a Calendar track, which appends its notes as steps in the workbook's order (a copy of it as it is then; notes already on the track are left where they are). It can also be added from a page on, for a workbook someone is partway through: the pages before that one are left out. A step added this way remembers its workbook (`CalendarEntries.workbook_id`), and opening it from the Calendar opens the note as a page of that workbook, with Previous and Next. It is only a pointer: deleting the workbook clears it, a workbook that no longer holds the note is ignored, and either way the note still opens. A note added to a track on its own has none, and a Kinpak does not carry it.

Both lists have a right-click menu. On a workbook: Open, Rename, *Share* or *Make Personal*, *Add to Calendar* (the whole workbook, onto a track picked from a submenu) and *Delete Workbook*. On a page of the open workbook: Open, the Calendar for *This Page* alone or *From Here On*, and *Take Out of Workbook*.

A Kinpak that carries shared notes carries the shared workbooks too (`workbook` items: a name and its notes' ids, in order), written after the notes they put in order. A workbook already in the workspace, by id, is the reader's and is never replaced.

14.5 Personal and shared

A note is personal or shared, chosen from the button under its title. Personal notes are the reader's own, like pins: always open to write, in a read-only workspace too, and never in a Kinpak or a sync. They are kept in a table of their own (`PersonalNotes`) rather than in a column of the shared one, so no export query can leak them by missing a filter. Shared notes (`Notes`) go into a Kinpak (*Shared notes*, on by default). Making a note shared asks first, since notes can be private thoughts.

Writing, editing, deleting and sharing shared notes is `allowEditSharedNotes`, which Read-only turns off. So in a read-only workspace a reader reads the shared notes and writes personal ones. A reader's shared notes cannot travel back to anyone until sync has collaborator licenses (Section 20).

14.6 Data

Both tables have the same shape and are `STRICT`, like every table:

Column	Meaning
`id`	A random ID (16 hex characters), so two devices can create notes without clashing and a rename keeps links working.
`title`	Empty while the note is Untitled.
`body`	The Markdown.
`drives`	The Drives it is filed under, newline-joined, as in `Glossary`.
`created_at`, `updated_at`	ISO times; the list is ordered by `updated_at`.

Notes are linked as `kinesis://note/<id>`. Both bodies are among the texts the link scan rewrites, so citations follow a renamed video and lose their link to a deleted one, and deleting a note turns links to it into plain words (the Trash puts them back). Renaming, moving or merging a Drive moves the notes filed under it, as it does terms, and a Drive with notes filed under it cannot be deleted. A Kinpak carries shared notes as `study_note` items, before the Calendar's tracks; a note already in the workspace is the reader's copy and is never replaced, and a track's entry for a note the pack does not carry (a personal one) arrives as a task named after it.

Notes are found from Ctrl+K (by title first, then by what they say), are a fourth kind of dot in the graph view, and are a folder of their own in the Obsidian export, where a citation's source becomes a wiki link to the video's note.

15 Logs

A log is a plain record of what was done, day by day (Figure 12): training, reading, a project. It is not the Calendar. Nothing in a log is planned, scheduled or ticked off; it is written after the fact, or just before.

A log is a name the reader gives it, and nothing else: it is not tied to a Drive. The list on the left ❷ shows each log with how many entries it has and when the last one was, the ones in use first with a dot and the empty ones dimmed with a ring, so nobody has to open a log to find out whether there is anything in it. *New Log* ❸ is at its foot. The open log's name ❹ is typed over to rename it, and it, *Today* ❺, *Go to: Top* and *Bottom* (the first and last rows showing), the bell and

delete stay at the top while the sheet scrolls under them. Right-clicking a log in the list has Open, Rename, *Remind Me Daily* and *Delete Log*; deleting asks first unless nothing is written in the log.

The open log is a sheet with three columns: Date, Entry and Description. The days are already there, a row each, the oldest first: thirty before today and thirty after, with more either way on request 8, and today's row tinted 6. There is nothing to add. Typing in a day's row is the entry, it is saved on leaving the row, and a row emptied of both its cells is removed. The date is the row's own and is never edited. A day is one row: more to say about it is another line in its description 7. The database holds it to that with a unique index on the log and the date, and writing a day is a single upsert (`write_log_day`): the row is made, changed, or, left empty, removed. *Today* brings today's row back to the middle of the pane, which is also where a log opens.

The description is plain text with a little emphasis (bold, italic, strikethrough and highlight, with the same keys as the Markdown editors) and nothing else. Right-clicking selected text there offers those four, then Cut, Copy and Paste, and none of the Markdown menu's headings, links or lists, which the cell would not draw. In particular nothing in a log links: a link typed there shows as the characters it was typed as, and the log tables are not among the texts the link scan reads or rewrites.

The search bar 1 filters the sheet to the rows whose description has what is typed. Three facets look elsewhere: `title_search:` (`!n`) narrows the list of logs by title, `entry_search:` (`!e`) looks in the Entry column, and `date_search:` (`!r`) in the date, as written (`2026-10-10`) or as shown (Oct 10).

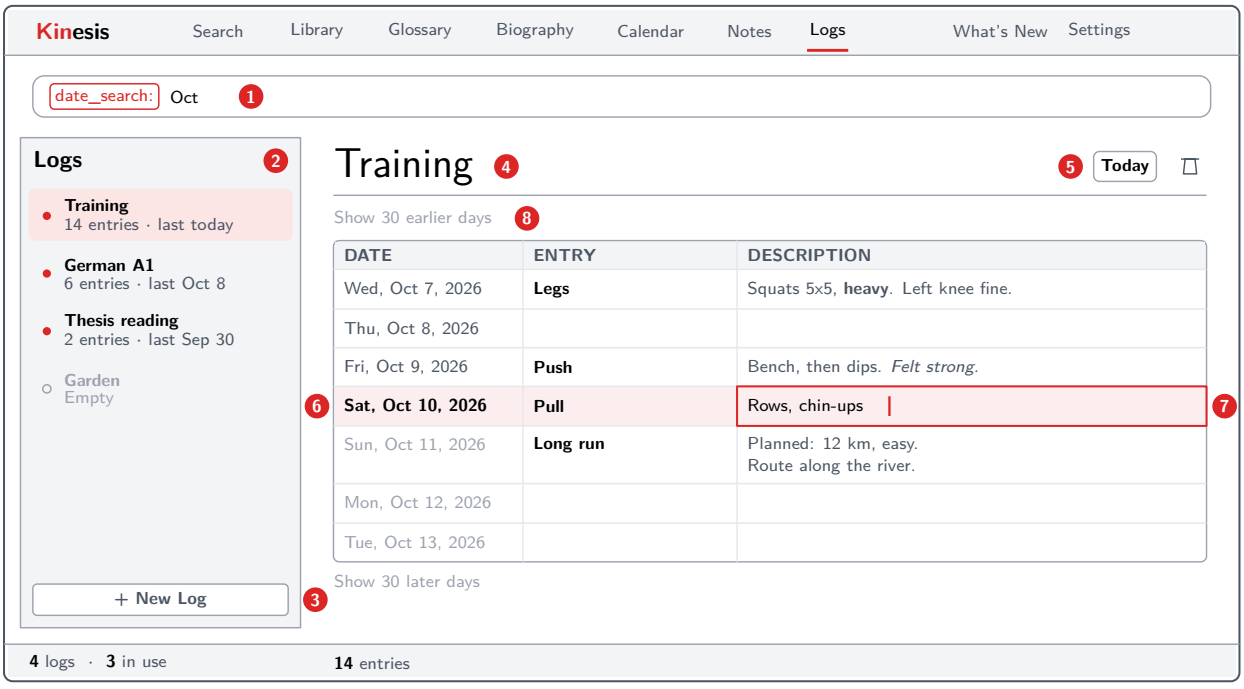


Figure 12: The Logs section, filtered by date, with today's row being written in.

A log can have an entry template: what each day's Entry starts as, so a routine need not be typed again every day. Its window has three tabs, one of which is in effect: *None*; *Every Day*, one text for any day; and *By Weekday*, a text for each weekday that should have one ("Push" on Mondays, "Rest" on Sundays), where a weekday left empty starts empty. `{{weekday}}` and `{{date}}` in a text are filled in for the day. It is set from the log's heading or its right-click menu.

The template is only what the sheet shows in a day with no row, dimmed: nothing is stored for a day until something is typed in its row, and then the Entry is saved with it. The pin menu's form starts from it too.

A log's bell asks for a reminder. With it on, a day on which the log has no row yet shows in the pin menu (Section 13): the log is marked under *Log Entry* and counted beside the pin, until today's row is written. Only today's row answers it; one written for another day does not. In a log with an entry template it is the row's description that answers it, since its Entry is there before anything is done; without one, either cell does. Any log can be written in from there too, bell or not: picking it under *Log Entry* gives today's row to fill in, in the menu.

Two tables, both `STRICT`: `Logs` (`id`, `name`, `created_at`, `remind`, and `entry_template`, the template as JSON) and `LogEntries` (`id`, `log_id`, `date`, `title`, `description`, and the two times), with the unique index `idxLogEntriesDay` on (`log_id`, `date`). The Entry column is `title` in the data; it is called Entry on screen because a "title" there was taken for the log's. A name is one log's, ignoring capitals. Logs are the reader's own, like pins and personal notes: they are not in a Kinpak and are not synced. The Obsidian export does write them (Section 19.2). The section is shown by the `showLogs` flag, and its name is the workspace's `aliasLogs` ("Tracker", say).

16 Graph view

Opened from the command palette (Ctrl+K), the graph view draws every video, term and person as a dot, joined where one links to another. Lines from each video to its creator can be turned on, but are off by default because they outnumber the links people actually wrote.

The layout is a force simulation written for the app. Repulsion uses a Barnes-Hut quadtree stored in flat typed arrays, and the simulation runs in a Web Worker so panning and zooming stay smooth while it settles. On a test graph of 8,188 dots, a step takes about 14 to 17 ms and the layout settles in three to four seconds. Drawing batches all lines and dots into a handful of canvas paths and skips anything off screen.

Opening something from the graph and then closing it, or pressing Back, returns to the graph as it was left.

Figure 13 shows the window. Videos are gray, terms amber and people blue, and each kind can be hidden ❶. *Creator lines* and *Unconnected* ❷ add the lines to each video's creator and the dots with no links at all; both start off, since they mostly add noise. *Find* centers the first dot whose name matches, and *Fit* brings everything back into view ❸. Pointing at a dot highlights its lines and shows a card with its name and how many connections it has ❹; clicking opens it.

17 Deleting and the Trash

Deleted videos, glossary entries and notes go to the Trash, which is kept in memory and never written to disk. Quitting the app or switching workspace empties it. This keeps deleted data out of sync and exports and stops it from piling up.

A delete takes more than one row with it, so the Trash keeps enough to put everything back: a video's row, its Drive links, its places in sequences, its note, its attachments, its pin, whether it was orphaned, and the links to it that were removed from other texts. Restoring something that has since been created again is refused. A note with nothing written in it is the one delete that

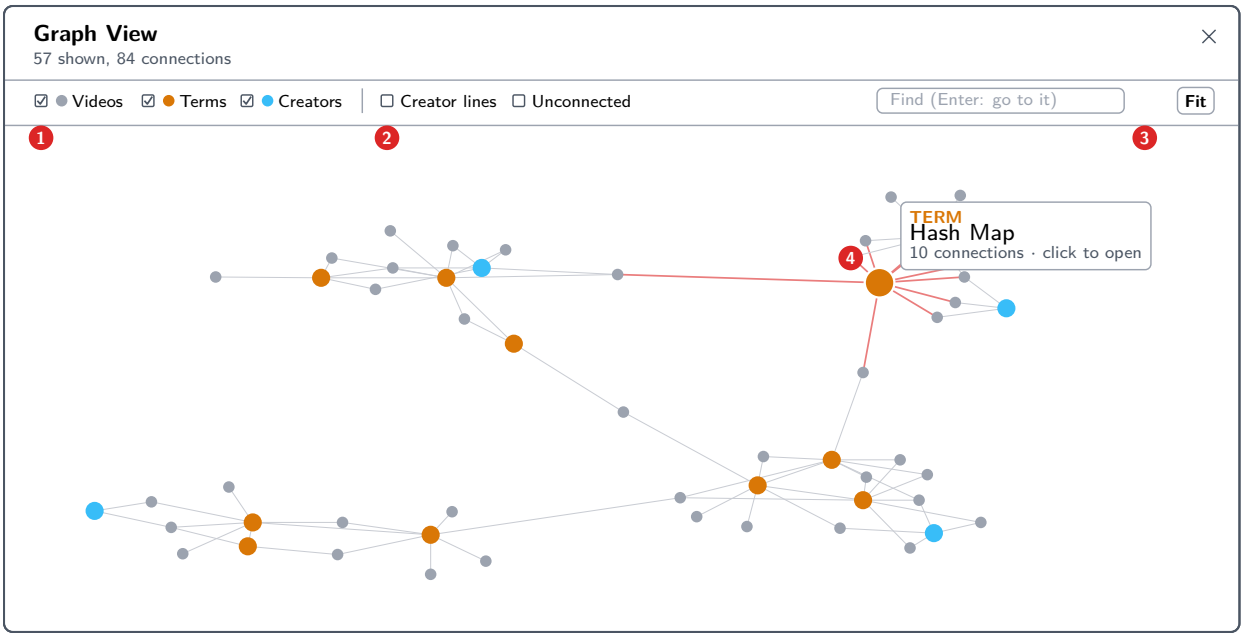


Figure 13: The graph view with a term hovered. Four topics form clusters, joined by the few links between them.

skips the Trash (Section 14). The Trash has a size limit, and the oldest items are dropped first.

18 Permissions

Every switch a workspace owner can set is a row in the `Settings` table: hiding a section or settings tab, turning off editing of summaries, Drives, glossary entries and so on. A missing or unreadable value uses its default, so a typo never turns a feature off by surprise. `docs/customizing.md` lists them all.

`workspaceReadOnly` turns off every content-editing permission at once without changing their individual values, so turning it off again restores the earlier setup. Read-only still allows what belongs to the reader rather than the workspace: checking off Calendar items and marking them in progress, adding and removing the five Essential tags, pinning, the Show Alias switch, and personal notes. The structure an owner built (tracks, custom tags, Drives, aliases) stays locked. Catching a renamed channel up with YouTube still happens in a read-only workspace, without the alias prompt, so a reader's copy does not drift from YouTube.

These checks are made in the interface. The backend has no read-only check of its own; a workspace that must not change should be distributed through a sync server, which can enforce settings.

19 Sharing and export

19.1 Kinpak

A Kinpak (`.kinpak`) is a workspace in one file: gzip-compressed newline-delimited JSON, using the same item format as the sync protocol. It starts with a header and the workspace's name, so the importer can show the name without reading the whole file. Then come the Drive tree, biographies, videos and their links, sequences, glossary, custom prompts, notes, attachments, search history, shared notes of the Notes section and the shared workbooks that order them, Calendar tracks, and finally the allowed settings.

- API keys, tokens, sync server details and folder paths are never written.
- Attachments are streamed in 4 MiB chunks in both directions, so no file has to fit in memory, and every file is checked against its hash on import.
- The file is written as `name.kinpak.part` and renamed when complete. A file with the real name is always a whole Kinpak.
- Drives can be left out. A plan worked out before writing decides what happens to everything that touches a left-out Drive: its videos, terms filed only there, creators with no remaining videos, and links pointing at any of these, which become plain text.

Importing into a workspace with the same name adds to it and never replaces the reader's own data. A note already written stays. A Calendar track whose name is already taken, or whose time would clash, is skipped and listed under "Left as they were" on the import screen. Calendar progress (what has been checked off) is a separate option, so a teacher can share a course without their own checkmarks. Pins, logs, and personal notes and workbooks are never included.

19.2 Obsidian

The Obsidian export writes a vault folder: one note per video in folders that mirror the Drive tree, one per glossary entry, one per biography, one per Calendar track with its entries as a checklist, one per note of the Notes section, and one per log. In-app links become wiki links, and a link to something left out of the export keeps only its text. A log is written as a table, the oldest day first, with each date as a wiki link by its ISO name (`[[2026-10-10]]`). That is what Obsidian's Daily Notes plugin calls a day's note, so with it on, a log's day opens that day's note, and the day's note lists under its backlinks the logs that have a row for it. The vault is the reader's own copy, so personal notes and logs are in it, which they never are in a Kinpak. The export always writes a new folder and never merges into an existing one.

20 Sync

The sync protocol (`docs/sync-protocol.md`) lets a server publish a curated library to many readers. Clients only pull. They ask for a manifest, then page through changes since their last revision. Each server reports an `epoch`; a client whose saved cursor belongs to another epoch does a full resync, because a rebuilt server's revision numbers can be higher than an old cursor and would otherwise look like a normal update.

A row is owned by the server if `SyncItems` has an entry for it. Only owned rows are ever deleted by a sync. When fields are merged, a non-null server value wins and a null one never erases a local value. Videos from a sync skip the normal save path, because its post-save steps rewrite content in ways that are not safe to repeat.

A server can also provide a license for Venice, YouTube and Pixabay. The client never receives the keys. It sends the provider request to the server's proxy with its sync token, and the server adds the key. In `fallback` mode a user's own key wins; in `enforce` mode the proxy is always used.

The reference server serves a read-only copy of a master database. The planned hosted setup uses Turso rather than a self-managed server, with separate license keys for regular readers and collaborators. That work has not started.

21 Interface conventions

A few conventions are applied throughout and are worth keeping when adding screens:

- Every main section has a bottom status bar.
- Dialogs use the shared `Modal` component, and closing one with unsaved changes asks first through the shared Unsaved Changes dialog.
- Common controls come from `src/components/ui`: `SecondaryButton` (a dialog or toolbar size), `Select` (a dropdown with its own chevron, since the browser's arrow ignores padding; every dropdown in the app uses it, in one of four looks) and `Segmented` (joined choices, such as Term and Definition in Flashcards). Classes passed to them are merged with `tailwind-merge`, so an override wins whatever order the CSS is in. Writing their classes out by hand is how small differences between screens creep in.
- Permanent deletes ask for confirmation unless the workspace turns that off. Deleting something with nothing in it (an empty note, a log with no entries) does not ask.
- A list's rows have a right-click menu (the shared `ContextMenu`, whose items can open one level of submenu) for what can be done to a row without opening it. Everything in such a menu can also be done from the opened thing's own page.
- Errors are shown inline or as a toast, never with a browser `alert()`.
- Colors come from theme variables, so every theme, light ones included, applies everywhere. The accent color marks what is selected or active; it is used sparingly.
- Back and Forward (Alt+Left and Alt+Right, or the mouse's side buttons) move through places visited: a section, a Drive, the video in the sidebar, and the graph view. Flashcards is not a place: a video, term or person opened from a card sets it aside, and closing that brings Flashcards back on the card it was left on. Typing in a search box updates the current place instead of adding a new one.

22 Testing

The Rust code has about 360 unit tests, run with `cargo test -lib`. Most open a fresh temporary database, so they test real SQL. The production schema tests build a database from

`production_schema.sql` and run the app against it end to end. The sync engine is tested against the real reference server.

The TypeScript side has a small Vitest suite (`npm test`) for the pure logic that a wrong edit would break quietly: reading a note's sources, Q&A blocks, callouts, flashcard decks, templates, and a track's repeat pattern against the same days the Rust tests expect. It is type-checked with `npx tsc -p tsconfig.app.json -noEmit`. Running `tsc -p .` checks nothing, because the root configuration only references the others.

23 Open work

The main piece of planned work is the hosted sync service described in the Sync section.